Libreto tecnico Streamlit

MÁSTER EN INTELIGENCIA ARTIFICIAL · UNIDAD 2

Libreto técnico de clase

Streamlit: el mostrador del modelo de gasto

Teoría oficial, arquitectura de ejecución y aplicación al proyecto de ventas. Material de estudio para 2 horas de clase y consulta posterior.

Carpeta aislada: `streamlit/`
Versión de la app: Streamlit 1.39.0 · scikit-learn 1.7.1 · joblib 1.5.1
Artefacto: predicción de *gasto a 90 días* a partir de un `.joblib` ya entrenado
Uso: leer antes o durante la clase; no sustituye ejecutar el notebook ni `streamlit run app.py`

Contenido

1. Cómo usar este libreto
2. Objetivo de la clase y qué no es
3. El problema de negocio
4. Entrenar no es servir
5. Dos puertas al mismo cerebro: FastAPI y Streamlit
6. Qué es Streamlit según la documentación oficial
7. Modelo de ejecución: el rerun
8. Widgets: pintan y devuelven un valor
9. Formularios: no predecir en cada tecla
10. Caché oficial: `cache_data` y `cache_resource`
11. Session State: memoria de una pestaña
12. Diseño de la página
13. El modelo de esta clase
14. Persistencia del modelo: joblib y scikit-learn
15. Mapa de la carpeta aislada
16. Recorrido de `app.py`
17. Lo que es de trabajo y lo que es atajo de clase
18. Arranque y comprobaciones
19. Glosario
20. Referencias oficiales
21. Ejercicios de estudio

1. Cómo usar este libreto

Este documento es el **material teórico oficial de la clase Streamlit**. Está escrito para que un alumno que no haya tocado una web en Python pueda seguirlo, y para que el docente lo use como guion de pizarra.

Tres capas, no se mezclan:

Capa	Dónde	Para qué
Teoría y contratos	este PDF	vocabulario, modelo mental, citas oficiales
Explicar y predecir sin UI	`notebooks/01_streamlit_prediccion.ipynb`	ejecutar `joblib.load` y `predict()`
La app de verdad	`app.py` / `app_hola.py`	`streamlit run` abre el navegador

La guía del docente (`GUIA_DOCENTE.md`) es el **minutado**. Este libreto es el **contenido**.

Regla de lectura: cada sección responde una pregunta de negocio o de ingeniería. Si no puedes decir en una frase *qué problema resuelve*, no pases de capítulo.

2. Objetivo de la clase y qué no es

Objetivo. Construir un *mostrador* (interfaz humana) delante de un modelo **ya entrenado**, de modo que negocio o el propio científico de datos pruebe predicciones sin Postman, sin JSON y sin Jupyter.

Qué se lleva el alumno

1. Distinguir *entrenar* (`fit`) de *servir* (`predict`).
2. Explicar el modelo de ejecución de Streamlit (rerun de todo el script).
3. Elegir `@st.cache_resource` para un modelo y `@st.cache_data` para un CSV.
4. Usar `st.form` para no disparar inferencia en cada tecla.
5. Servir **el mismo** `.joblib` que FastAPI, por otra puerta.

Qué no es esta clase

- No es una clase de Machine Learning: no afinamos hiperparámetros ni el MAE.
- No es una clase de frontend (no hay React, HTML ni CSS que el alumno deba escribir).
- No es un sustituto de FastAPI: Streamlit no es una API REST.
- No se sube un Excel nuevo ni se descarga un modelo de internet.

3. El problema de negocio

La plataforma de ventas ya tiene histórico. En unidades anteriores se persistió, se midió calidad, se hizo SQL avanzado y se leyó a escala con Spark. En FastAPI se **sirvió** el modelo a *otro programa*.

El hueco que queda: un responsable de CRM, un docente o un analista quiere preguntar

> «Si este cliente tiene 52 compras y 23 601 € de historial, ¿cuánto estimamos que gastará en 90 días?»

sin escribir un `POST` ni un notebook. Eso es un **data app**: una aplicación pequeña, hecha por quien entiende el dato, para *usar* un modelo o un análisis.

Streamlit se posiciona exactamente ahí. La documentación oficial de *Get started* lo presenta como la forma de construir *data apps*: mostrar datos, dibujar gráficos, añadir widgets, cachear cómputo y definir temas (docs.streamlit.io/get-started).

En esta clase el *data app* no analiza un CSV suelto: **pregunta a un cerebro ya entrenado**.

4. Entrenar no es servir

```
ENTRENAR (offline, una vez)          SERVIR (online, muchas veces)
ventas.csv → features → fit()        cargar el archivo UNA vez
           → archivo .joblib         cada clic / cada POST solo predict()
```

Momento	Quién	Qué ocurre	Dónde en esta carpeta
Entrenar	script, no la app	`fit()`, escribe disco	`scripts/entrenar_modelos.py`
Cargar	arranque de la app	`joblib.load`	`src/carga_modelos.py` + caché
Inferir	cada interacción	`modelo.predict(x)`	`src/prediccion.py`

La app **nunca** llama a `fit()`. Si el `.joblib` no está, el error debe ser claro: «ejecuta el script», no un `FileNotFoundError` a mitad de un clic.

Ese dibujo es el mismo de FastAPI. Cambia solo *quién pulsa el botón*.

5. Dos puertas al mismo cerebro: FastAPI y Streamlit

	FastAPI (clase anterior)	Streamlit (esta clase)
Cliente	Otro programa, otra API, un móvil	Una persona en el navegador
Idioma	HTTP + JSON	Widgets (slider, formulario, pestaña)
Proceso	`uvicorn api.main:app --port 8000`	`streamlit run app.py` (puerto 8501)
Contrato de entrada	Pydantic (`EntradaPrediccion`)	`min_value` , tipos del widget, `st.form`
Carta	`/docs` (OpenAPI / Swagger)	La propia página
Cocinero	`modelos/*.joblib`	**El mismo archivo** (copia local)

Frase de clase. *FastAPI habla JSON. Streamlit habla sliders. El cerebro es el mismo archivo.*

FastAPI sigue siendo la pieza correcta cuando un microservicio, un ERP o un batch necesita predicciones. Streamlit es la pieza correcta cuando un humano tiene que *ver y tocar* el modelo. En un trabajo real conviven: la API en producción, el mostrador Streamlit para *demo, QA y negocio*.

Esta carpeta está **aislada** a propósito: no importa `api/` ni el `src/` del proyecto padre. El `.joblib` se copió. El alumno no depende de Uvicorn hoy.

6. Qué es Streamlit según la documentación oficial

Definición operativa (documentación de Streamlit, *Get started* y *Fundamentals*):

> Streamlit es un framework de Python para convertir un script en una aplicación web de datos. El desarrollador escribe Python; el framework pinta la interfaz y vuelve a ejecutar el script cuando el usuario interactúa.

Implicaciones que hay que saber de memoria:

1. **No hay plantillas HTML.** `st.title` , `st.slider` , `st.dataframe` *son* la interfaz.
2. **El archivo es el programa.** Se lanza con `streamlit run archivo.py` , no con el botón Run de Jupyter.
3. **El servidor abre el navegador** en `http://localhost:8501` por defecto.
4. **Cada interacción rerunea el script** de arriba abajo (capítulo 7).

5. Está pensado para *data apps* (tablas, gráficos, modelos), no para un e-commerce con carrito y login complejo. Se puede, pero no es su fuerte.

Instalación oficial: entorno virtual + `pip install streamlit`. En esta clase el pin está en `requirements.txt` (`streamlit==1.39.0`) para que todos vean lo mismo.

Documentación de partida: docs.streamlit.io/get-started.

7. Modelo de ejecución: el rerun

Esta es la idea que, si no se entiende, hace que el resto parezca magia.

La documentación de *execution flow* y *session state* lo dice así:

> Por defecto, las apps Streamlit ejecutan el script **entero**. Cada vez que el usuario interactúa, Streamlit vuelve a correr el archivo de la primera línea a la última. Cada rerun parte de una pizarra en blanco: las variables locales **no** se comparten entre ejecuciones.

Fuente: docs.streamlit.io/develop/api-reference/execution-flow y docs.streamlit.io/develop/concepts/architecture/session-state.

Consecuencias prácticas:

Si haces esto	Pasa esto
`modelo = joblib.load(...)` al inicio, sin caché	Cada slider relée el archivo. Lento e inútil
`contador = 0` y luego `contador += 1`	En el siguiente rerun `contador` vuelve a 0
Mueves un slider	Se pinta otra vez el título, la barra lateral, las pestañas
`st.button("Saludar")`	Devuelve `True` **solo** en el clic; al siguiente rerun es `False`

Por eso existen tres mecanismos oficiales que *rompen* la pizarra en blanco de forma controlada:

1. **Estado de los widgets** — Streamlit recuerda el valor del slider él solo.
2. **`st.session_state`** — tú guardas variables entre reruns (por pestaña del navegador).
3. **Caché (`cache_data` / `cache_resource`)** — resultados o recursos **compartidos entre usuarios**, no solo entre clics.

`st.rerun()` fuerza un rerun inmediato. `st.stop()` corta el script. `@st.fragment` permite rerunear *solo un trozo* (avanzado; no lo necesitamos hoy).

Experimento de clase. Lanza `app_hola.py`, mueve el slider y mira la terminal: Streamlit reimprime el script. Aún no hay modelo. Solo se ve el rerun.

8. Widgets: pintan y devuelven un valor

Un widget hace **dos cosas a la vez**: dibuja un control y **devuelve** a Python el valor actual. No hay un `onChange` obligatorio como en JavaScript clásico: el valor *es* la variable.

```
gasto = st.slider("Gasto histórico (€)", min_value=0, max_value=20000, value=2000)
# gasto es un int. Si el alumno mueve la barra, el script se relanza y gasto cambia.
```

Catálogo mínimo de esta clase (API oficial de widgets: docs.streamlit.io/develop/api-reference/widgets):

Método	Qué pinta	Qué devuelve	Uso en la clase
`st.text_input`	caja de texto	`str`	`app_hola.py` (nombre)
`st.number_input`	número con flechas	`int` o `float`	las 4 features
`st.slider`	barra	número	demo del rerun
`st.radio`	opciones excluyentes	el elemento elegido	bosque vs red
`st.selectbox`	desplegable	el elemento elegido	`cliente_id`
`st.button`	botón	`True` solo en el clic	saludar
`st.form_submit_button`	botón de formulario	`True` al enviar el form	predecir
`st.title` / `st.write`	título / texto o markdown	nada	cabecera
`st.metric`	KPI grande	nada	valor predicho
`st.dataframe`	tabla	nada (en esta clase)	ficha del cliente
`st.bar_chart`	barras	nada	comparar modelos
`st.success` / `st.info` / `st.error`	caja de color	nada	mensajes

`value=` es el **arrastre inicial**, no un valor que se pisa en cada rerun. Una vez el usuario cambia el widget, Streamlit conserva *ese* valor (estado del widget).

`key=` identifica un widget cuando hay dos iguales en la misma página (en `app.py` el `cliente_id` de la pestaña Comparar lleva `key="cliente_cmp"` para no chocar con el de la pestaña 2).

`min_value=0` en los `number_input` es el **portero pobre**: no es Pydantic, pero impide un gasto negativo desde la UI. La validación seria de producción sigue siendo un contrato (Pydantic en la API).

9. Formularios: no predecir en cada tecla

Documentación oficial (*Group multiple widgets*):

> Por defecto Streamlit rerunea el script en cada interacción. A veces es mejor esperar a que un grupo de widgets esté relleno. Eso es `st.form`.

Fuente: docs.streamlit.io/develop/api-reference/execution-flow.

```
with st.form("form_manual"):
    # estos widgets NO disparan rerun al teclear
    lanzar = st.form_submit_button("Predecir gasto a 90 días")

if lanzar:
    valor = predecir(paquete, **cifras)
```

Reglas oficiales que importan hoy:

- Solo `st.form_submit_button` puede llevar *callback* dentro de un form.
- Los widgets del form **sí** se pueden leer y escribir vía Session State.
- El modelo **no corre** mientras el alumno cambia `num_compras`. Corre al pulsar.

En la pestaña «Ficha a mano» de `app.py` el form existe precisamente para no llamar a `predict()` en cada tecla. En la pestaña «Elegir un cliente» no hace falta: cambiar el id *sí* debe recalcular (un clic, una predicción).

10. Caché oficial: `cache_data` y `cache_resource`

Documentación: docs.streamlit.io/develop/concepts/architecture/caching.

> La caché guarda resultados de llamadas lentas para no repetirlas. Los valores cacheados están disponibles para **todos los usuarios** de la app. Si el resultado debe ser solo de *esta* sesión, usa Session State.

Hay **dos** decoradores. La guía oficial pide no usar el antiguo `@st.cache`.

10.1 `st.cache_data` — datos

Para valores **serializables**: DataFrames, números, dicts, resultados de una query, transformaciones.

- Devuelve una **copia**. Mutar el DataFrame que te dieron no ensucia el caché.
- Si cambian los argumentos de la función, se recalcula.
- En esta clase: el lookup CSV y el `meta.json`.

```
@st.cache_data
def _lookup() -> pd.DataFrame:
    return cargar_lookup()
```

10.2 `st.cache_resource` — recursos globales

Para objetos que **no quieres copiar ni recargar**: modelos de ML, conexiones a base de datos, clientes HTTP pesados.

La documentación lo dice con el ejemplo de un *pipeline* de inferencia: *«the recommended way to cache global resources like ML models or database connections»*.

- No copia el objeto: es un **singleton**.
- Una mutación afecta a todos los usuarios. El objeto debe ser *thread-safe*.
- Un `RandomForestRegressor` o un `Pipeline` de sklearn **no se mutan** al predecir: `predict()` no cambia los árboles. Es el caso de uso canónico.

```
@st.cache_resource
def _modelos() -> dict:
    return {
        "sklearn": cargar_paquete("gasto_sklearn.joblib"),
        "mlp": cargar_paquete("gasto_mlp.joblib"),
    }
```

10.3 Cómo decidirlo en un examen oral

Pregunta	Decorador
¿Es un DataFrame, un JSON, una lista?	`cache_data`
¿Es un modelo, un engine de SQL, un cliente?	`cache_resource`
¿Debe ser distinto por alumno / pestaña?	no es caché: `session_state`

Sin caché, cada rerun haría `joblib.load` (~decenas o cientos de ms, y peor si el archivo crece). Con caché, el primer usuario paga la carga; el resto reutiliza el objeto en RAM.

11. Session State: memoria de una pestaña

Documentación: docs.streamlit.io/develop/concepts/architecture/session-state.

> Una *sesión* es una pestaña del navegador conectada al servidor. Session State comparte variables **entre reruns de esa sesión**. No se comparte entre pestañas ni entre personas.

```
if "ultimo_cliente" not in st.session_state:
    st.session_state.ultimo_cliente = 8
st.session_state.ultimo_cliente = cliente_id
```

Límites oficiales que debes contar:

- Recargar la pestaña (F5) o seguir un enlace markdown **tira** el WebSocket y resetea el estado.

- No es una base de datos. No auditamos predicciones ahí.
- El estado de los widgets ya vive en la sesión; Session State es para *lo que tú inventas* (historial de predicciones de la demo, un flag, un contador).

En `app.py` **no hace falta** Session State para la demo de 2 horas: los widgets recuerdan solos y el modelo está en `cache_resource`. El ejercicio plus del notebook (guardar el último saludo de `app_hola.py`) existe solo para *ver* el mecanismo.

12. Diseño de la página

API de layout (oficial): *sidebar*, *columns*, *tabs*, `set_page_config`.

Método	Efecto	En `app.py`
`st.set_page_config`	pestaña del navegador, `layout="wide"`	**solo una vez y al principio** del script
`st.sidebar`	columna izquierda	radio del motor + ficha `meta.json`
`st.tabs`	pestañas; cada `with` es una	manual / cliente / comparar
`st.columns(n)`	parte la fila	4 inputs; 3 métricas
`st.divider`	línea	`app_hola.py`

`layout="wide"` usa el ancho del monitor: útil cuando hay tabla + métricas. `app_hola.py` usa `centered` porque es una demo de un widget.

No hay CSS que el alumno deba escribir. Si se quiere marca, Streamlit admite temas en `.streamlit/config.toml` (fuera de alcance hoy).

13. El modelo de esta clase

13.1 Qué predice

Un **número**: el gasto estimado del cliente en los **90 días siguientes** al corte `2025-06-30`. No predice si compra, ni el producto, ni un *churn*.

	Nombre	Significado
X	`num_compras`, `gasto_historico`, `ticket_medio`, `cantidad_total`	Resumen del **pasado** (compras con `fecha_venta ≤ 2025-06-30`)
y	`gasto_siguiente_periodo`	Suma de `total` entre el 1-jul y el 28-sep-2025. Si no compró: `0`

El orden de las 4 columnas es un **contrato**. Si se permutan, el `.joblib` no avisa: predice basura. `src/carga_modelos.py` comprueba que `paquete["features"] == COLUMNAS_FEATURES`.

13.2 De dónde salen las cifras

`src/features_modelo.py` (copia local) aplica la misma idea del snapshot SQL:

- **Pasado** → lo que el modelo tiene derecho a ver.
- **Futuro** → el target. El modelo **no lo ve** en `predict()`.

`ventas.csv` está copiado en `data/raw/` solo para **reentrenar** si alguien borra `modelos/`. La app **no** lee `ventas.csv`. Lee el lookup ya agregado.

13.3 Los dos cerebros

Archivo	Algoritmo	Nota
`gasto_sklearn.joblib`	`RandomForestRegressor` (40 árboles, profundidad 8)	scikit-learn clásico
`gasto_mlp.joblib`	`Pipeline(StandardScaler + MLPRegressor(16, 8))`	red pequeña; el scaler va *dentro* del archivo

Misma X, misma y, dos puertas en la barra lateral. El MAE ronda 1 000 € y el R^2 ~0,05–0,06: **hoy no importa**. Hoy se enseña a servir, no a ganar un ranking.

El cliente **8** (gran comprador, target real ≈ 1 956 €) y el **9** (no volvió, target 0) son los dos ejemplos de pizarra.

13.4 Qué hay dentro del `.joblib`

No es «el modelo suelto». El script guardó un **diccionario**:

```
{
    "modelo": objeto_sklearn,
    "features": ["num_compras", "gasto_historico", "ticket_medio", "cantidad_total"],
    "target": "gasto_siguiente_periodo",
    "version": "1.0.0",
    "metricas": {"mae": ..., "r2": ..., "segundos": ...},
}
```

`predict()` exige una matriz (1 fila × 4 columnas), no un dict. Por eso existe `src/prediccion.py`: un solo sitio que arma el `DataFrame` con `columns=COLUMNAS_FEATURES`.

14. Persistencia del modelo: joblib y scikit-learn

Documentación oficial de scikit-learn, *Model persistence*:

> Tras entrenar, se quiere **guardar** el modelo y **cargarlo** en otro proceso sin volver a `fit()`. scikit-learn recomienda `joblib` (más eficiente que `pickle` para arrays grandes de NumPy).

Fuente: scikit-learn.org/stable/model_persistence.html. joblib: joblib.readthedocs.io.

Reglas que hay que decir en voz alta:

1. El archivo es **binario**. No se abre en Excel.
2. `joblib.dump` / `joblib.load` deben usarse con **la misma familia de versión** de scikit-learn. Por eso `requirements.txt` pinea `scikit-learn==1.7.1`. Si un alumno instala sklearn 1.3, el `load` puede fallar o advertir.
3. Cargar en cada petición es un antipatrón. En FastAPI se carga en el `lifespan`. En Streamlit, en `@st.cache_resource`.
4. En producción el archivo suele vivir en S3, Azure Blob o MLflow, no en la carpeta del repo. El *gesto* es el mismo: `load` una vez, `predict` muchas.

15. Mapa de la carpeta aislada

```
streamlit/
├── notebooks/01_streamlit_prediccion.ipynb   guion de las 2 horas
├── app_hola.py                               widgets, sin modelo
├── app.py                                    mostrador de predicción
├── src/
│   ├── features_modelo.py                    contrato X / y
│   ├── carga_modelos.py                      joblib.load + lookup
│   └── prediccion.py                         1 fila → número
├── scripts/entrenar_modelos.py               solo si faltan artefactos
├── modelos/                                  cerebro + fichas (ya copiados)
├── data/raw/ventas.csv                       copia; solo para reentrenar
├── requirements.txt
└── docs/                                     este libreto + guía docente
```

Archivo	Pregunta que responde
`features_modelo.py`	¿Qué ve el modelo y qué intenta acertar?
`entrenar_modelos.py`	¿De dónde nace el `.joblib`?
`carga_modelos.py`	¿Cómo se descongela sin romper el contrato?
`prediccion.py`	¿Cómo se arma la única fila de `predict()`?
`app_hola.py`	¿Qué es un widget y el rerun?
`app.py`	¿Cómo lo usa un humano?
`features_clientes.csv`	¿Cuál es la ficha de cada `cliente_id`?

Nada de esta carpeta importa PostgreSQL, Spark ni Uvicorn.

16. Recorrido de `app.py`

Orden de lectura en el editor (el mismo orden que el rerun):

1. `set_page_config` — primero, una vez.
2. **Funciones cacheadas** — `_modelos`, `_lookup`, `_meta`. Se definen y luego se llaman. El primer rerun carga disco; los siguientes tocan RAM.
3. **Cabecera** — deja claro que no hay `fit()`.
4. **Sidebar** — elige el cerebro (`sklearn` / `mlp`) y enseña MAE y fecha de corte. El MAE alto es didáctico: «hoy servimos».
5. **Pestaña 1 · Ficha a mano** — el humano escribe las 4 cifras. Equivale al `POST /predecir/sklearn` de FastAPI. `st.form` + `predecir`.
6. **Pestaña 2 · Elegir un cliente** — equivale al `GET /predecir/cliente/{id}`. Las 4 cifras salen del CSV. Se muestra predicho, real y error absoluto. El target **no entra** al modelo.
7. **Pestaña 3 · Comparar** — misma ficha, dos `.joblib`, un `st.bar_chart`. El contrato no cambia; cambia el archivo.

`predecir` no sabe si el paquete es bosque o red. Ese es el punto de ingeniería: **un contrato, N cerebros**.

17. Lo que es de trabajo y lo que es atajo de clase

Pieza	¿Producción?	Comentario
Entrenar offline y dejar un artefacto	Sí	MLflow / S3 en vez de la carpeta `modelos/`
Cargar el modelo una vez	Sí	`cache_resource` o el `lifespan` de FastAPI
Widgets + form para un humano	Sí	demos, herramientas internas, QA
CSV congelado de junio como «base de clientes»	No	la ficha debe estar viva (DB, feature store, o la manda el cliente)
MLP de scikit-learn como «la red»	Gesto sí	en un banco sería ONNX, PyTorch, SageMaker…
Sin autenticación ni log de predicciones	No	Streamlit Community Cloud / SSO / tabla `predicciones`
Servir *solo* con Streamlit a 10 000 peticiones/s	No	eso es la API (FastAPI, BentoML, un model server)

Frase. *En producción el camarero para máquinas es la API. Streamlit es la sala de catas. El vino es el `.joblib`.*

18. Arranque y comprobaciones

Desde **esta** carpeta, con `py_env`:

```
conda activate py_env
cd ...\persisitencia\streamlit
pip install -r requirements.txt
streamlit run app_hola.py
streamlit run app.py
```

El notebook se abre *después* del `pip`. No instala paquetes él solo.

Si falta un `.joblib`:

```
python scripts/entrenar_modelos.py
```

Comprobación rápida sin UI (la misma que el Bloque 1 del notebook): cargar el paquete, ficha del cliente 8, `predecir`. Si eso funciona, la app solo es pintura.

Puerto por defecto: **8501**. No choca con el 8000 de Uvicorn. Se pueden tener los dos abiertos: misma predicción, dos puertas.

19. Glosario

Término	Definición en esta clase
Inferencia	Usar un modelo ya entrenado para obtener un número nuevo
Artefacto	El archivo que deja el entrenamiento (`.joblib`)
Rerun	Volver a ejecutar el script Streamlit de arriba abajo
Widget	Control visual que **devuelve** un valor Python
Contrato	Nombres y orden de columnas que el modelo espera
Lookup	Tabla `cliente_id → 4 cifras` (CSV, no la DB)
Target	Lo que el modelo intenta acertar; no se le pasa en `predict()`
`cache_resource`	Singleton en RAM para el modelo
`cache_data`	Copia cacheada de un DataFrame / JSON
Session State	Memoria *por pestaña*, entre reruns
FastAPI	Puerta máquina–máquina (JSON)
Streamlit	Puerta humano–modelo (widgets)
MAE	Error absoluto medio; hoy no se optimiza

20. Referencias oficiales

Documentación que este libreto resume. Conviene abrirlas en clase al menos una vez.

Streamlit

- Get started: https://docs.streamlit.io/get-started
- Fundamentals (modelo mental de *data app*): https://docs.streamlit.io/get-started/fundamentals
- Execution flow (rerun, forms, `st.stop`, fragments): https://docs.streamlit.io/develop/api-reference/execution-flow
- Caching overview (`cache_data` vs `cache_resource`): https://docs.streamlit.io/develop/concepts/architecture/caching
- Caching and state (API): https://docs.streamlit.io/develop/api-reference/caching-and-state
- Session State: https://docs.streamlit.io/develop/concepts/architecture/session-state
- Widgets: https://docs.streamlit.io/develop/api-reference/widgets
- `st.cache_resource` (ejemplo oficial: cargar un modelo): https://docs.streamlit.io/develop/api-reference/caching-and-state/st.cache_resource

Modelo y serialización

- scikit-learn · Model persistence: https://scikit-learn.org/stable/model_persistence.html
- joblib: https://joblib.readthedocs.io/en/stable/
- `RandomForestRegressor`: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html
- `MLPRegressor`: https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPRegressor.html

La otra puerta (clase anterior)

- FastAPI: https://fastapi.tiangolo.com/
- Pydantic: https://docs.pydantic.dev/

21. Ejercicios de estudio

1. Explica, sin código, por qué un `joblib.load` al nivel del módulo *sin* `@st.cache_resource` es un error de diseño.
2. ¿Qué decorador usarías para cachear el resultado de `predict()` de mil clientes? ¿Y para el objeto del modelo? Justifica con la documentación oficial.
3. Cambia en `app.py` el `value` inicial de `gasto_historico` a 5 000. ¿Cambia la predicción de un alumno que *ya* había movido el widget? ¿Por qué?
4. Cliente 9: anota predicho bosque, predicho red y target. ¿Qué le dirías a negocio?
5. Añade un `st.slider` de `num_compras` en la pestaña manual, *dentro* del form. ¿Cuántos reruns hay hasta pulsar el botón?
6. (Plus) En `app_hola.py`, guarda el último saludo en `st.session_state`. Recarga la pestaña. ¿Sigue ahí? Relaciónalo con el WebSocket de la documentación.

Fin del libreto. La clase no termina al leer: termina cuando el alumno ha lanzado `streamlit run app.py`, ha predicho el cliente 8 a mano y por id, y ha comparado los dos `.joblib` con la misma ficha.